Лабораторная работа № 2

Тема: «Функции, области видимости, файловая система и оценка сложности алгоритмов»

Среда выполнения: Google Colab + GitLab

Прикладная задача: «Аналитика интернет-магазина» — построить модуль чтения, фильтрации, агрегации и сохранения данных о заказах интернет-магазина с использованием функций, областей видимости, файловых операций и оценки сложности алгоритмов.

Часть I. Теория

1. Функции и процедуры

В Python **нет строгого разделения** на функции и процедуры. На практике придерживаются соглашения:

Термин	Определение	Возвращает	Пример
Функция	Возвращает результат через return	Значение	def add(a, b): return a + b
Процедура	Не возвращает явного значения	None	def print_greeting(name): print(f"Привет, {name}")

1.1. Синтаксис определения функции

```
def имя_функции(аргументы) -> тип_возврата:
    """Docstring — краткое описание."""
    # тело функции
    return результат
```

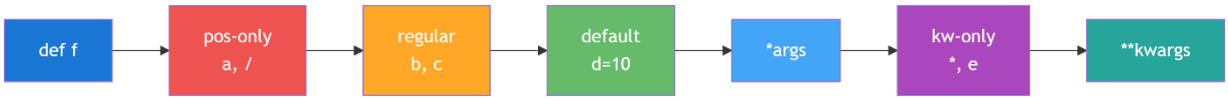
1.2. Примеры

```
def print_greeting(name: str) -> None:
    """Процедура: выводит приветствие."""
    print(f"Привет, {name}!")
def add(a: int, b: int) -> int:
    """Функция: возвращает сумму двух чисел."""
    return a + b
```

Важно: Аннотации типов не проверяются во время выполнения, но их понимают IDE, mypy и читатели кода. В современном Python — стандарт.

2. Аргументы функций

Python поддерживает **шесть видов** параметров. Их порядок в определении функции строго фиксирован.



2.1. Сравнительная таблица

Вид	Синтаксис	Что передаётся	Пример вызова
Позиционный	def f(a, b)	По порядку	f(1, 2)
Именованный	def f(a, b)	По имени	f(b=2, a=1)
По умолчанию	def f(a, b=10)	Опускается при вызове	f(1)
Positional-only	def f(a, /, b)	Только по позиции	f(1, 2)
Keyword-only	def f(*, key)	Только по имени	f(key=1)
*args	def f(*args)	Все позиционные → tuple	f(1, 2, 3)
**kwargs	def f(**kwargs)	Все именованные → dict	f(a=1, b=2)

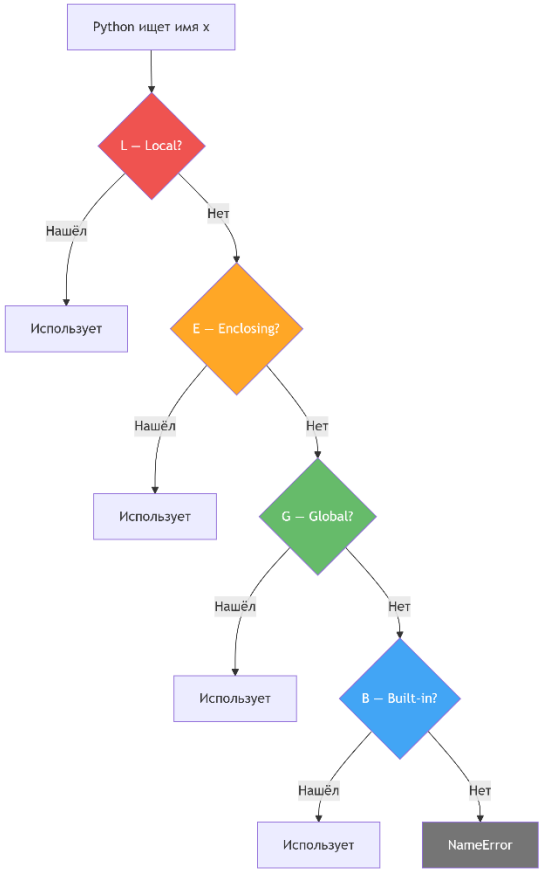
2.2. Пример: все виды аргументов сразу

```
def full(pos_only, /, regular, *args, kw_only, **kwargs):
    print("pos_only:", pos_only)
    print("regular:", regular)
    print("args:", args)
    print("kw_only:", kw_only)
    print("kwargs:", kwargs)
full(1, 2, 3, 4, kw_only=5, extra=6)
# pos_only: 1
# regular: 2
# args: (3, 4)
# kw_only: 5
# kwargs: {'extra': 6}
```

Подводный камень: никогда не используйте **изменяемые** значения (например, [] или {}) как значения по умолчанию. Список создаётся один раз и будет «накапливаться» между вызовами.

3. Области видимости и правило LEGB

Когда Python встречает имя переменной, он ищет его в **строгом порядке** (правило LEGB).



3.1. Расшифровка уровней

Уровень	Где	Пример
Local	Внутри текущей функции	Параметры, локальные переменные
Enclosing	Во внешней (вложенной) функции	Переменные родительской функции
Global	На уровне модуля	Глобальные переменные, функции
Built-in	Встроенные имена Python	len, print, range

3.2. Ключевые слова global и nonlocal

```
counter = 0
def increment():
```

```
    global counter        # изменяем глобальную переменную
    counter += 1
    return counter
def outer():
    x = 1
    def inner():
        nonlocal x        # изменяем переменную enclosing-функции
        x += 1
        return x
    return inner()
```

3.3. Замыкания

Замыкание — функция, которая «запоминает» переменные из enclosing-области, даже если внешняя функция завершилась.

```
def make_counter():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    return inner
c1 = make_counter()
c2 = make_counter()
print(c1(), c1())  # 1 2 — свой счётчик
print(c2())        # 1 — независимый
```

4. Работа с файловой системой

4.1. Модуль os: основные функции

Функция	Назначение	Пример
os.getcwd()	Текущий рабочий каталог	/content
os.listdir(path)	Список файлов и папок	['orders.txt', 'src']
os.mkdir(path)	Создать папку	os.mkdir("new")
os.makedirs(path)	Создать папку со всеми родительскими	os.makedirs("a/b/c")

Функция	Назначение	Пример
os.remove(path)	Удалить файл	os.remove("a.txt")
os.rmdir(path)	Удалить **пустую** папку	os.rmdir("empty")
os.rename(old, new)	Переименовать / переместить	os.rename("a.txt", "b.txt")
os.path.join(...)	Соединить части пути	os.path.join("data", "orders.txt")
os.path.exists(path)	Проверить существование	True / False
os.path.basename(path)	Имя файла	"data/x.txt" → "x.txt"
os.path.dirname(path)	Каталог	"data/x.txt" → "data"
os.path.splitext(path)	Имя + расширение	"x.txt" → ("x", ".txt")

4.2. Открытие файлов

```
python
# Чтение
with open("orders.txt", "r", encoding="utf-8") as f:
    text = f.read()

# Запись (перезапись)
with open("orders.txt", "w", encoding="utf-8") as f:
    f.write("Новое содержимое")

# Дозапись
with open("orders.txt", "a", encoding="utf-8") as f:
    f.write("\nДобавленная строка")
```

4.3. Режимы открытия файла

Режим	Назначение	Создаёт?	Перезаписывает?
r	Чтение	Нет	—
w	Запись	Да	Да
a	Дозапись в конец	Да	Нет
x	Создание нового	Да (ошибка, если есть)	—
r+	Чтение и запись	Нет	Частично
rb / wb	Бинарный режим	—	—

Важно: Всегда используйте with open(...) as f: — файл закроется автоматически даже при исключении. И всегда указывайте encoding="utf-8" для текстовых файлов.

5. Оценка сложности алгоритмов

Асимптотическая сложность — это характеристика роста времени работы алгоритма при увеличении размера входных данных. Обозначается через **О-нотацию** (Big O).

5.1. Основные классы сложности

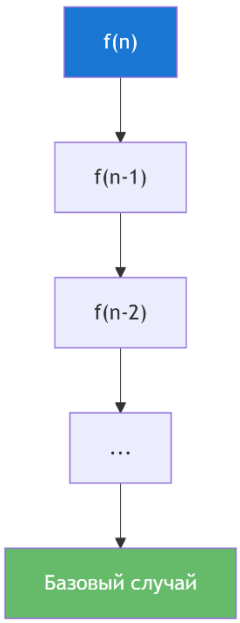
Класс	Название	Пример алгоритма
O(1)	Константная	Доступ к элементу по индексу
O(log n)	Логарифмическая	Бинарный поиск
O(n)	Линейная	Линейный поиск, суммирование
O(n log n)	Линейно-логарифмическая	Сортировка слиянием, sorted
$O(n^2)$	Квадратичная	Сортировка пузырьком, вложенные циклы
$O(2^n)$	Экспоненциальная	Наивные числа Фибоначчи

Класс	Название	Пример алгоритма
O(n!)	Факториальная	Перебор перестановок

5.2. Правила оценки

Правило	Пример	Результат
Сокращение констант	O(5n)	O(n)
Сложение последовательных операций	O(n) + O(n)	O(n)
Умножение вложенных циклов	O(n) × O(n)	$O(n^2)$
Оставить наибольший порядок	$O(n^2 + n)$	$O(n^2)$

5.3. Сложность рекурсивных алгоритмов



Структура рекурсии	Рекуррентное соотношение	Сложность	Пример
Один вызов f(n-1)	T(n) = T(n-1) + O(1)	O(n)	Факториал, сумма цифр

Структура рекурсии	Рекуррентное соотношение	Сложность	Пример
Один вызов f(n/2)	T(n) = T(n/2) + O(1)	O(log n)	Бинарный поиск
Два вызова f(n-1), f(n-1)	T(n) = 2T(n-1) + O(1)	$O(2^n)$	Наивный Фибоначчи
Два вызова f(n/2)	T(n) = 2T(n/2) + O(n)	O(n log n)	Merge sort

5.4. Сравнение: рекурсия vs итерация

Аспект	Рекурсия	Итерация
Читаемость	Часто выше	Иногда ниже
Память	O(глубина) (стек вызовов)	O(1)
Риск	RecursionError при большой глубине	Нет
Сложность	Легко ошибиться с базой	Прозрачна

6. Рекурсия

Рекурсия — приём, при котором функция вызывает саму себя. Обязательные компоненты:

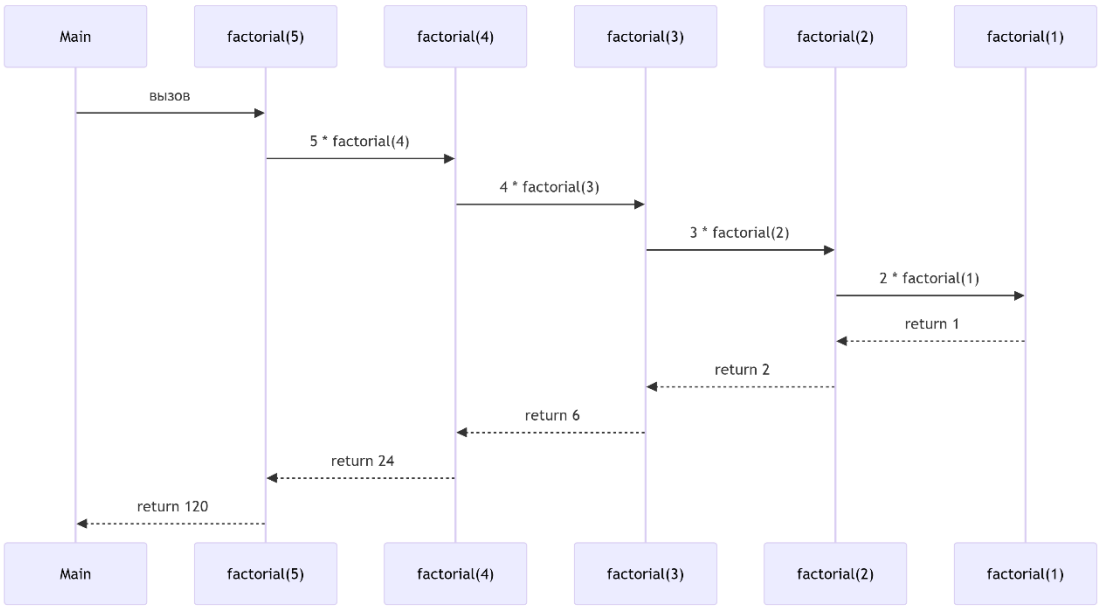
1. **Базовый случай** — условие выхода из рекурсии.
2. **Рекурсивный шаг** — вызов функции с меньшим (приближающимся к базе) аргументом.

6.1. Классические примеры

```
def factorial(n: int) -> int:
    """Факториал: n! = 1 * 2 * ... * n."""
    if n <= 1:
        return 1                    # базовый случай
    return n * factorial(n - 1)     # рекурсивный шаг
def fib(n: int) -> int:
    """Наивный Фибоначчи (O(2^n))."""
    if n <= 1:
```

```
        return n
    return fib(n - 1) + fib(n - 2)
def sum_digits(n: int) -> int:
    """Сумма цифр числа."""
    if n < 10:
        return n
    return n % 10 + sum_digits(n // 10)
```

6.2. Стек вызовов



Подводный камень: Python по умолчанию ограничивает глубину рекурсии ~1000 вызовов (sys.getrecursionlimit()). Для глубокой рекурсии используйте итерацию или увеличьте лимит (sys.setrecursionlimit).

Примеры

Общие примеры

Пример 1. Функции и процедуры

```
def print_greeting(name: str) -> None:
    """Процедура: выводит приветствие."""
    print(f"Привет, {name}!")


def add(a: int, b: int) -> int:
    """Функция: возвращает сумму двух чисел."""
    return a + b
print_greeting("Мир")
result = add(2, 3)
print("Результат:", result, "| тип:", type(result))
```

Пример 2. Все виды аргументов

```
def power(base: float, exp: float = 2) -> float:
    """Возведение в степень (по умолчанию — квадрат)."""
    return base ** exp
print(power(3))              # 9
print(power(3, 3))           # 27
print(power(exp=3, base=3))  # 27
def sum_all(*args: float) -> float:
    """Сумма произвольного числа аргументов."""
    return sum(args)
print(sum_all(1, 2, 3, 4))   # 10
def print_info(**kwargs) -> None:
    """Выводит именованные аргументы."""
    for key, value in kwargs.items():
        print(f"  {key}: {value}")

print_info(name="Анна", age=25, city="Москва")
```

Пример 3. Область видимости: global и nonlocal

```
counter: int = 0
def increment() -> int:
    global counter
    counter += 1
    return counter
increment()
increment()
print("counter =", counter)  # 2
def outer() -> int:
    x: int = 1
    def inner() -> int:
        nonlocal x
        x += 1
        return x
    return inner()
print("outer() =", outer())  # 2
```

Пример 4. Рекурсия: факториал

```
python
import sys
```

```
def factorial(n: int) -> int:
    """Факториал через рекурсию. Сложность: O(n) по времени и памяти."""
    if n <= 1:
        return 1
    return n * factorial(n - 1)
print("5! =", factorial(5))
print("10! =", factorial(10))
print("Лимит рекурсии Python:", sys.getrecursionlimit())
def factorial_iter(n: int) -> int:
    """Факториал через цикл. Сложность: O(n) времени, O(1) памяти."""
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
print("Итеративный 10! =", factorial_iter(10))
```

Пример 5. Модуль os: базовые операции

```
import os
print("Текущая директория:", os.getcwd())
print("Файлы в текущей папке:", os.listdir(".")[:5], "...")
path = os.path.join("data", "orders.txt")
print("Собранный путь:", path)
print("Существует ли путь:", os.path.exists(path))
print("Имя файла:", os.path.basename(path))
print("Директория:", os.path.dirname(path))
print("Без расширения:", os.path.splitext(os.path.basename(path))[0])
```

Пример 6. Чтение и запись файла

```
import os
# Запись
with open("demo.txt", "w", encoding="utf-8") as f:
    f.write("Строка 1\nСтрока 2\nСтрока 3")
# Чтение
with open("demo.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print("Содержимое:")
    print(content)
# Чтение построчно
```

```
with open("demo.txt", "r", encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(f"Строка {i}: {line.strip()}")
os.remove("demo.txt")  # очистка
```

Пример 7. Измерение сложности: линейный vs бинарный поиск

```
import time
import random
n = 1_000_000
data = sorted(random.sample(range(n * 10), n))
target = data[-1]  # худший случай
def linear_search(arr, t):
    for i, x in enumerate(arr):
        if x == t:
            return i
    return -1
def binary_search(arr, t):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == t:
            return mid
        elif arr[mid] < t:
            left = mid + 1
        else:
            right = mid - 1
    return -1
t0 = time.perf_counter()
linear_search(data, target)
t1 = time.perf_counter()
print(f"Линейный поиск: {t1 - t0:.6f} с")
t0 = time.perf_counter()
binary_search(data, target)
t1 = time.perf_counter()
print(f"Бинарный поиск: {t1 - t0:.6f} с")
```

Примеры для прикладной задачи

В качестве прикладной задачи рассмотрим **«Аналитику интернет-магазина»**. У нас есть файл orders.txt, в котором построчно записаны заказы в формате:

```
order_id,customer_id,product,unit_price,quantity,status
ORD-1000,cust_001,Apple Laptop,350000,2,completed
ORD-1001,cust_005,Samsung Phone,85000,1,shipped
```

Нужно прочитать данные, отфильтровать по статусу, посчитать выручку, отсортировать топ-товары и сохранить сводный отчёт.

Пример 8. Создание тестового файла с заказами

```
import os
import random
random.seed(42)
os.makedirs("lab2_data", exist_ok=True)
BRANDS = ["Apple", "Samsung", "Lenovo", "Asus", "Sony"]
CATEGORIES = ["Laptop", "Phone", "Tablet", "Accessories"]
STATUSES = ["completed", "shipped", "pending", "cancelled"]
lines = ["order_id,customer_id,product,unit_price,quantity,status"]
for i in range(200):
    brand = random.choice(BRANDS)
    category = random.choice(CATEGORIES)
    price = random.randint(5000, 250000)
    qty = random.randint(1, 5)
    status = random.choice(STATUSES)
    cust = f"cust_{random.randint(1, 30):03d}"
    lines.append(
        f"ORD-{1000 + i},{cust},{brand} {category},{price},{qty},{status}"
    )
with open("lab2_data/orders.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(lines))
print(f"Файл создан: lab2_data/orders.txt")
print(f"Всего заказов: {len(lines) - 1}")
```

Пример 9. Чтение заказов из файла

```
def read_orders(filename: str) -> list[dict]:
    """Читает заказы из файла, возвращает список словарей."""
    orders: list[dict] = []
    with open(filename, "r", encoding="utf-8") as f:
        # пропускаем заголовок
        next(f)
        for line in f:
            line = line.strip()
```

```
            if not line:
                continue
            order_id, cust_id, product, price, qty, status = line.split(",")
            orders.append({
                "order_id": order_id,
                "customer_id": cust_id,
                "product": product,
                "unit_price": int(price),
                "quantity": int(qty),
                "status": status,
            })
    return orders

orders = read_orders("lab2_data/orders.txt")
print(f"Загружено заказов: {len(orders)}")
print("Первый заказ:", orders[0])
```

Пример 10. Фильтрация и агрегация

```
python
def filter_by_status(orders: list[dict], status: str) -> list[dict]:
    """Возвращает заказы с заданным статусом."""
    return [o for o in orders if o["status"] == status]

def total_revenue(orders: list[dict]) -> int:
    """Считает суммарную выручку заказов."""
    return sum(o["unit_price"] * o["quantity"] for o in orders)

completed = filter_by_status(orders, "completed")
print(f"Завершённых заказов: {len(completed)}")
print(f"Суммарная выручка: {total_revenue(completed):,} руб.")
```

Пример 11. Сохранение результатов

```
def save_report(
    filename: str,
    orders: list[dict],
    total: int,
) -> None:
    """Сохраняет сводный отчёт по заказам."""
    with open(filename, "w", encoding="utf-8") as f:
```

```
        f.write(f"Всего заказов: {len(orders)}\n")
        f.write(f"Общая выручка: {total:,} руб.\n")
        f.write("Топ-5 заказов:\n")
        top5 = sorted(
            orders,
            key=lambda o: o["unit_price"] * o["quantity"],
            reverse=True,
        )[:5]
        for o in top5:
            amount = o["unit_price"] * o["quantity"]
            f.write(f"  {o['order_id']} | {o['product']} | {amount:,} руб.\n")


os.makedirs("lab2_output", exist_ok=True)
save_report("lab2_output/report.txt", completed, total_revenue(completed))
print("Отчёт сохранён: lab2_output/report.txt")
```

Пример 12. Запуск и демонстрация

```
def main() -> None:
    orders = read_orders("lab2_data/orders.txt")
    completed = filter_by_status(orders, "completed")
    total = total_revenue(completed)
    save_report("lab2_output/report.txt", completed, total)
    print("Готово. Проверьте lab2_output/report.txt")
if __name__ == "__main__":
    main()
```

Часть II. Задания

Задания на освоение концепции (1–10)

Задание 1. Функция vs процедура.
Напишите:

1. **Процедуру** print_order_summary(order: dict) -> None, которая выводит информацию о заказе в формате "ORD-1000: Apple Laptop × 2 = 700 000 руб. [completed]".
2. **Функцию** order_total(order: dict) -> int, которая возвращает сумму заказа (unit_price × quantity).

Проверьте обе на нескольких заказах. Объясните в комментарии, чем процедура отличается от функции.

Задание 2. Все виды аргументов.
Напишите функцию describe_order(order_id: str, status: str = "pending", *tags: str, priority: int = 1, **extra: object) -> None, которая:

1. Печатает ID заказа и статус.
2. Печатает список тегов (каждый на новой строке).
3. Печатает приоритет.
4. Печатает дополнительные параметры.

Продемонстрируйте **три разных вызова** с разными комбинациями аргументов. Объясните, что попадёт в tags и extra.

Задание 3. Значения по умолчанию и ловушка изменяемых объектов.

1. Реализуйте функцию add_order(order, orders=[]), которая добавляет заказ в список. Продемонстрируйте **баг накопления** между вызовами.
2. Исправьте функцию, используя None как значение по умолчанию и создавая новый список внутри.
3. Напишите тест, который доказывает, что исправленная версия работает корректно.

Задание 4. Области видимости: global **и** nonlocal**.**

1. Создайте **глобальный счётчик** processed_count = 0 и функцию process_order(order) -> dict, которая увеличивает счётчик через global и возвращает обработанный заказ.
2. Создайте фабрику счётчиков make_revenue_counter() с замыканием и nonlocal — она возвращает функцию, которая при каждом вызове прибавляет сумму заказа к внутреннему итогу и возвращает накопленную выручку. Создайте **два независимых** счётчика.
3. Объясните в комментарии, **почему** global **считается антипаттерном** и когда nonlocal оправдан.

Задание 5. Рекурсия: сумма цифр в ID заказа.

1. Напишите **рекурсивную** функцию sum_digits(n: int) -> int — сумму цифр числа.
2. Напишите **итеративную** версию той же функции.
3. Оцените сложность обеих версий в терминах Big O (время и память).
4. Примените: извлеките числовую часть из "ORD-12345" и найдите сумму цифр 12345. Замерьте время для 500-значного числа — какая версия быстрее и почему?

Задание 6. Файловая система: вывод списка .txt**-файлов.**

1. Используя os.listdir и os.path.splitext, выведите список всех файлов в текущей директории с расширением .txt.
2. Модифицируйте код: рекурсивно обойдите все подпапки через os.walk и соберите все .txt-файлы.
3. Для каждого файла выведите его полный путь и размер в байтах (os.path.getsize).

Задание 7. Создание папки с проверкой.
Напишите функцию create_folder(path: str) -> bool, которая:

1. Создаёт папку, если её ещё нет (используйте os.makedirs(..., exist_ok=True)).
2. Возвращает True, если папка была создана, False — если уже существовала.
3. Проверьте функцию на трёх разных путях: существующий, новый, вложенный (a/b/c).

Задание 8. Подсчёт строк в файле заказов.
Напишите функцию count_orders(filename: str) -> int, которая:

1. Возвращает количество **непустых** строк в файле (без заголовка).
2. Обрабатывает FileNotFoundError: если файла нет — возвращает -1.
3. Использует контекстный менеджер with.

Продемонстрируйте работу на созданном файле lab2_data/orders.txt.

Задание 9. global **vs** nonlocal **в контексте магазина.**

1. Реализуйте **функцию-фабрику** make_discount_manager() — она возвращает функцию, которая принимает сумму заказа, накапливает общую сумму и возвращает итог с накопленной скидкой 5 % (при сумме > 100 000). Используйте nonlocal.
2. Реализуйте ту же задачу через **класс** (просто для сравнения, ООП будет в ЛР4).
3. Объясните, почему замыкание с nonlocal иногда удобнее класса, а иногда — нет.

Задание 10. Сравнение сложности: линейный vs бинарный поиск в заказах.

1. Сгенерируйте список из **1 000 000** числовых ID заказов (случайные числа) и отсортируйте его.
2. Напишите **линейный** поиск linear_search(arr, target) -> int — O(n).
3. Напишите **бинарный** поиск binary_search(arr, target) -> int — O(log n).
4. Замерьте время для 1000 поисков каждого метода. Во сколько раз бинарный поиск быстрее? Объясните через Big O.

Задания по прикладной задаче (11–20)

Прикладная задача: «Аналитика интернет-магазина».

Интернет-магазин хранит данные о заказах в текстовом файле. Формат — строки вида:

```
order_id,customer_id,product,unit_price,quantity,status
ORD-1000,cust_001,Apple Laptop,350000,2,completed
ORD-1001,cust_005,Samsung Phone,85000,1,shipped
```

где поля:

- order_id — уникальный идентификатор заказа;

- customer_id — идентификатор клиента;
- product — название товара;
- unit_price — цена за единицу, руб.;
- quantity — количество, шт.;
- status — статус (completed, shipped, pending, cancelled).

Постановка задачи. Требуется построить **модуль аналитики**, который:

- читает данные о заказах из файла;
- фильтрует и сортирует заказы;
- считает статистику (суммарная выручка, топ-товары, топ-клиенты, распределение по статусам);
- сохраняет результаты в папку lab2_output/;
- использует **функции, области видимости, файловые операции, рекурсию и оценку сложности**.

Инициализация данных (одинакова для всех заданий 11–20).

Запустите эту ячейку один раз — данные будут использоваться во всех заданиях.

```
import os
import random
random.seed(42)
# Создаём рабочие директории
os.makedirs("lab2_data", exist_ok=True)
os.makedirs("lab2_output", exist_ok=True)
# Генерируем тестовый файл с заказами
BRANDS = ["Apple", "Samsung", "Lenovo", "Asus", "Sony"]
CATEGORIES = ["Laptop", "Phone", "Tablet", "Accessories"]
STATUSES = ["completed", "shipped", "pending", "cancelled"]
lines = ["order_id,customer_id,product,unit_price,quantity,status"]
for i in range(500):
    brand = random.choice(BRANDS)
    category = random.choice(CATEGORIES)
    price = random.randint(5000, 250000)
    qty = random.randint(1, 5)
    status = random.choice(STATUSES)
    cust = f"cust_{random.randint(1, 40):03d}"
    lines.append(
        f"ORD-{1000 + i},{cust},{brand} {category},{price},{qty},{status}"
    )
ORDERS_FILE = os.path.join("lab2_data", "orders.txt")
```

```
with open(ORDERS_FILE, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))
print(f"[Успешно] Файл создан: {ORDERS_FILE}")
print(f"Всего заказов: {len(lines) - 1}")
print("Первые 3 строки данных:")
for line in lines[1:4]:
    print(" ", line)
```

Задание 11. Чтение заказов из файла.
Напишите функцию read_orders(filename: str) -> list[dict], которая:

1. Читает файл lab2_data/orders.txt.
2. Возвращает список словарей вида:

{"order_id": str, "customer_id": str, "product": str,

"unit_price": int, "quantity": int, "status": str}

3. Обрабатывает ошибки:
    - FileNotFoundError — вернуть пустой список и вывести предупреждение.
    - Некорректные строки (не 6 полей, нечисловые unit_price/quantity) — пропустить с предупреждением.
4. Использует аннотации типов и docstring.

Задание 12. Фильтрация заказов.
Напишите набор функций:

1. filter_by_status(orders: list[dict], status: str) -> list[dict] — оставляет заказы с заданным статусом.
2. filter_by_min_total(orders: list[dict], min_total: int) -> list[dict] — оставляет заказы с суммой не меньше порога (unit_price × quantity >= min_total).
3. filter_completed(orders: list[dict]) -> list[dict] — заказы только со статусом completed.

Используйте генераторные выражения. Каждая функция — одна ответственность.

Задание 13. Сортировка заказов по ключу.
Напишите функцию sort_orders(orders: list[dict], key: str, reverse: bool = False) -> list[dict], которая:

1. Сортирует заказы по одному из ключей: "unit_price", "quantity", "total" (вычисляемый unit_price × quantity), "customer_id".
2. Использует sorted с key=lambda ....
3. Если передан несуществующий ключ — выбрасывает ValueError.
4. Замерьте сложность сортировки: O(n log n).

Задание 14. Сохранение заказов в файл.
Напишите функцию save_orders(orders: list[dict], filename: str) -> None, которая:

1. Сохраняет список заказов в файл **в том же формате**, что и входной (с заголовком).
2. Создаёт директории, если их нет (os.makedirs).
3. Использует with open(..., "w", encoding="utf-8").

4. Сохраните завершённые заказы (status == "completed") в lab2_output/completed.txt.

Задание 15. Группировка заказов по клиентам.
Напишите функцию group_by_customer(orders: list[dict]) -> dict[str, list[dict]], которая:

1. Возвращает словарь: customer_id → список заказов.
2. Использует dict.setdefault или collections.defaultdict.
3. Отсортируйте словарь по убыванию **суммарной выручки** клиента.
4. Выведите топ-5 клиентов с их выручкой в читаемом виде.

Задание 16. Удаление отменённых заказов.
Напишите функцию remove_cancelled(orders: list[dict]) -> list[dict], которая:

1. Возвращает **новый** список без заказов со статусом cancelled (не мутирует исходный).
2. Использует генератор списка.
3. Проверьте: исходный список orders не изменился после вызова.
4. Замерьте время: за сколько обрабатывается список из 500 000 заказов? Объясните через O(n).

Задание 17. Создание директории и сохранение отчёта.
Напишите функцию save_report(content: str, filename: str) -> str, которая:

1. Создаёт директорию lab2_output/reports/, если её нет.
2. Сохраняет content в файл lab2_output/reports/<filename>.
3. Возвращает **полный путь** к сохранённому файлу (через os.path.abspath).
4. Продемонстрируйте: сохраните строку "Отчёт по аналитике магазина" в файл summary.txt и распечатайте путь.

Задание 18. Композиция: read + filter + save.
Напишите функцию load_and_process(input_file: str, status: str, output_file: str) -> int, которая:

1. Читает заказы через read_orders (задание 11).
2. Фильтрует через filter_by_status (задание 12).
3. Сохраняет через save_orders (задание 14).
4. Возвращает **количество** сохранённых заказов.
5. Запустите функцию на lab2_data/orders.txt с status="completed" и output_file="lab2_output/processed.txt".

Задание 19. Рекурсивный поиск самого дорогого заказа.

1. Напишите **рекурсивную** функцию max_order_recursive(orders: list[dict]) -> dict, которая находит заказ с максимальной суммой (unit_price × quantity) без использования встроенного max().
2. Оцените сложность: T(n) = T(n-1) + O(1) → O(n).
3. Сравните с итеративной версией через замер времени на списке из 10 000 заказов.
4. Продемонстрируйте: найдите топ-заказ и выведите его order_id, product и сумму.

Задание 20. Комплексный отчёт: единый pipeline.
Соберите все функции из заданий 11–19 в **единый модуль**:

1. Функция generate_report(input_file: str, output_dir: str) -> dict:
    - читает заказы;
    - считает: total_orders, total_revenue, completed_count, cancelled_count, avg_order_total;
    - строит top_5_orders (задание 19 — используйте итеративную версию для скорости);
    - строит top_5_customers (задание 15);
    - сохраняет completed.txt (задание 14) и summary.txt (задание 17);
    - возвращает словарь со статистикой.
2. Функция main() -> None — оркестратор:
    - вызывает generate_report для lab2_data/orders.txt;
    - красиво печатает результаты;
    - указывает пути к созданным файлам.
3. **Точка входа:** используйте идиому if __name__ == "__main__": main().
4. Все функции должны быть с аннотациями типов и docstring.
5. Проверьте, что код проходит flake8 и mypy (если установлены).

Критерии оценки:

- Нет дублирования (DRY).
- Каждая функция делает **одну** вещь (SRP).
- Есть обработка ошибок.
- Соблюдён PEP 8.
- Сложность ключевых операций оценена и обоснована.

Контрольные вопросы

1. Чем функция отличается от процедуры в Python?
2. Что такое область видимости? Какие виды областей видимости существуют в Python?
3. Для чего используются ключевые слова global и nonlocal? Приведите примеры, где каждое из них необходимо.
4. Какие функции модуля os вы знаете? Приведите примеры.
5. Как открыть файл для чтения и записи? Какие режимы открытия бывают? Чем опасен режим w без проверки?
6. Что такое асимптотическая сложность алгоритма? Приведите примеры O(1), O(n), $O(n^2)$.
7. В чём особенность рекурсивных алгоритмов? Как оценить их сложность?
8. Как связаны рекурсия и стек вызовов? Что произойдёт при слишком глубокой рекурсии?
9. Почему **изменяемые значения** (например, []) опасны как значения по умолчанию для аргументов функции?

10. Почему binary_search работает за O(log n), а linear_search — за O(n)? Приведите пример из прикладной задачи.

Дополнительно

1. Оформите выполненные задания из ноутбука в виде отдельного Jupyter-ноутбука с именем lab2_functions_files.ipynb. В ноутбуке должны быть:
    - титульный markdown-блок с указанием ФИО, группы и номера лабораторной работы;
    - отдельные ячейки с решениями заданий 1–20;
    - комментарии к коду (что делает функция, какие аргументы принимает, что возвращает).
2. Убедитесь, что ноутбук полностью выполняется сверху вниз без ошибок: последовательно запустите все ячейки через меню **Kernel → Restart & Run All**. При необходимости исправьте порядок ячеек и зависимости между ними.
3. Создайте в своём аккаунте на GitLab новый приватный репозиторий с именем python-labs (если он ещё не создан) и добавьте в него папку lab2.
4. Инициализируйте в папке lab2 локальный git-репозиторий, если его нет:

```
cd lab2
git init
```

5. Добавьте в папку lab2 следующие файлы:
    - lab2_functions_files.ipynb — итоговый ноутбук с решениями;
    - orders.txt — входные данные (создаётся в примере 8);
    - report.txt — результаты обработки (создаётся в примере 11);
    - README.md — краткое описание работы: цель, какие задания выполнены, какие функции реализованы, оценка сложности основных алгоритмов.
6. Создайте файл .gitignore, исключив из отслеживания служебные и сгенерированные файлы:

```
lab2_data/
lab2_output/
.ipynb_checkpoints/
__pycache__/
*.pyc
```

7. Зафиксируйте изменения и отправьте их в удалённый репозиторий:

```
git add .
git commit -m "ЛР2: функции, области видимости, файлы, сложность"
git push origin main
```

8. Убедитесь, что после git push файлы действительно появились в удалённом репозитории (откройте страницу репозитория в браузере).
9. В конце README.md добавьте раздел **«Ссылка на репозиторий»** и укажите полный URL вашего репозитория.

10. Ссылку на репозиторий и сам ноутбук прикрепите в качестве ответа на задание в системе дистанционного обучения.